

Name: _____

ERP: _____

Question:	Tracing	Short Questions	Implementing ADTs	Inheritance	Total
Marks:	16	22	12	20	70
Score:					

Question 1: Tracing 16 marks

(a) [2 marks] Which class destructor will be called first, when **a**, **b**, and **c** go out of scope?

```

1 class A { };
2 class B { };
3 class C: public B { };
4 int main(){
5     A a;
6     B b;
7     C c;
8 }
```

Solution: C (in reverse order of construction)

(b) [3 marks] What will be the output of following?

```

1 #include <iostream>
2 using namespace std;
3 class Base {
4     public:
5         virtual void print() const {
6             cout << "Base Class";
7         }
8 };
9 class Derived : public Base {
10     public:
11         void print() const override {
12             cout << "Derived Class";
13         }
14 };
15 void display(const Base& obj) {
16     obj.print();
17 }
18
19 int main() {
20     Base b;
21     Derived d;
22     display(b);
23     display(d);
24 }
```

Solution: Base Class
Derived Class

(c) [3 marks] What will be the output of following?

```

1  #include <iostream>
2  using namespace std;
3  class A {
4      public:
5          virtual void display() {
6              cout << "A";
7          }
8  };
9  class B : public A {
10     public:
11         void display() override {
12             cout << "B";
13         }
14 };
15 class C : public B {
16     public:
17         void display() override {
18             cout << "C";
19         }
20 };
21 int main() {
22     A a;    B b;    C c;
23     A* ptr1 = &b;
24     B* ptr2 = &c;
25     ptr1->display();
26     ptr2->display();
27     return 0;
28 }

```

Solution: BC

(d) [3 marks] what will be output of following program?

```

class Point {
    public:
        Point() { cout << "Normal Constructor called\n"; }
        Point(Point &t) { cout << "Copy constructor called\n"; }
};

int main() {
    Point *t1, *t2;
    t1 = new Point();
    t2 = new Point(*t1);
    Point t3 = *t1;
    Point t4; t4= t3;
    Point t5 = t3;
}

```

Solution:

Normal Constructor called
Copy constructor called
Copy constructor called
Normal Constructor called
Copy constructor called

- (e) [5 marks] Analyze the code and predict the output. If an exception occurs, identify the type of exception and explain the control flow.

```
1  class CustomException : public std::exception {
2      public:
3          const char* what() const throw() {
4              return "Custom Exception";
5          }
6  };
7  void processNumber(int number) {
8      try {
9          if (number < 0)
10             throw CustomException();
11          else if (number % 2 == 0)
12             throw "Even Number";
13          else
14             throw 42.42;
15      } catch (const char* msg) {
16          std::cout << "Caught Exception: " << msg << std::endl;
17      } catch (double value) {
18          std::cout << "Caught Exception: " << value << std::endl;
19      } catch (...) {
20          std::cout << "Caught Unknown Exception" << std::endl;
21          throw; // Rethrow the unknown exception
22      }
23  }
24  int main() {
25      try {
26          processNumber(-5);
27          processNumber(8);
28          processNumber(17);
29      } catch (const std::exception& e) {
30          std::cerr << "Caught Standard Exception: " << e.what() << std::endl;
31      } catch (...) {
32          std::cerr << "Caught Unknown Exception in main" << std::endl;
33      }
34      return 0;
35  }
```

Solution: In the first call to `processNumber`, `CustomException` is thrown, caught by the catch-all block in `processNumber`, and rethrown. The rethrown exception is caught by the `catch(std::exception&)` block in `main`. The rest of the code in `main` is not executed. The

following output is produced:

Caught Unknown Exception

Caught Standard Exception: Custom Exception

Question 2: Short Questions22 marks

- (a) [4 marks] Match the following C++ keywords to the most appropriate description below: **class**, **public**, **void**, **main**, **private**, **this**, **nullptr**, **new**

Write *one* keyword in each blank and *each keyword will be used exactly once*.

public	defines something as being part of the API
new	creates an instance
nullptr	the value for pointer variables when they do not point to an instance
private	hidden from clients
main	method name that is called when the program is run
void	return type of a method that does not return any value
class	contains definitions of methods and fields
this	refers to the instance upon which the current method or constructor acts

- (b) [5 marks] The following program with declaration and usage of the class **Clock** contain some errors. The data members and definitions of the member functions are irrelevant for this question and are therefore omitted. Find 5 lines containing errors. Each line number can be specified only once, even if the line contains several errors (of the same type).

```

1  class Clock {
2
3      // creates a clock showing time h:m:s (h < 24, m < 60, s < 60)
4      Clock (unsigned int h, unsigned int m, unsigned int s) { /* omitted */ }
5
6      // advances the clock's time by one second
7      void tick() const { /* omitted */ }
8
9      // h, m, s are set to the clock's hour, minute and second
10     void time (unsigned int h, unsigned int m, unsigned int s) const
11
12     { /* omitted */ }
13
14     private:
15         // data members to store the time /* omitted */
16 };

```

```

17 main() {
18     Clock c = (22, 59, 59);
19     c.tick();
20     unsigned int h; unsigned int m; unsigned int s;
21     time (h, m, s); // 23:00:00
22     return 0;
23 }

```

Line number	Error
Line # 2	The constructor, timer , and tick functions should be declared public .
Line # 7&10	Since time and tick member functions meant to change the state of the object, they should not be declared const .
Line # 17	Return type of main should be int
Line # 18	Incorrect syntax. With paranthesis initialization, there should be no = operator.
Line # 21	Member function time can only be called from an object.

(c) [6 marks] Read the following program and answer the questions below.

```

1  struct Box {
2      int val;
3      Box* a;
4      Box* b;
5  };
6  int main() {
7      Box* x = new Box;
8      Box* y = new Box;
9      x->val = 0;
10     x->a = x;
11     x->b = y;
12     y->val = 1;
13     y->a = x;
14     y->b = nullptr;
15
16     Box* tmp = x;
17     x = y;
18     y = tmp;
19
20     y->b->a = x->a->b;
21 }

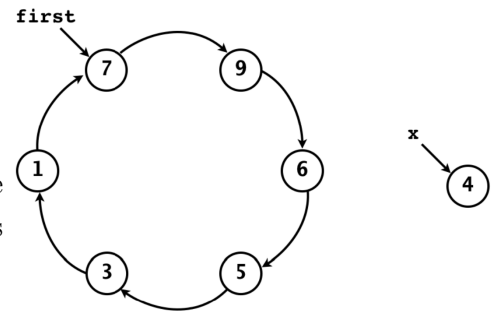
```

- After line 14 executes, what is the value of **x->b->val**? Write the value here: 1
- After line 18 executes, what is the value of **x->a**? Check one of the 3 boxes below.
☒ Refers to the **Box** with val 0 ☐ Refers to the **Box** with val 1 ☐ nullptr
- After line 18 executes, what is the value of **x->b**? Check one of the 3 boxes below.
☐ Refers to the **Box** with val 0 ☐ Refers to the **Box** with val 1 ☒ nullptr
- After line 18 executes, what is the value of **y->a**? Check one of the 3 boxes below.
☒ Refers to the **Box** with val 0 ☐ Refers to the **Box** with val 1 ☐ nullptr
- After line 18 executes, what is the value of **y->b**? Check one of the 3 boxes below.
☐ Refers to the **Box** with val 0 ☒ Refers to the **Box** with val 1 ☐ nullptr
- Which field changes when line 20 executes? Check one of the 5 boxes below.
☒ **x->a** ☐ **x->b** ☐ **y->a** ☐ **y->b** ☐ n/a (segmentation fault)

(d) [7 marks] Suppose that the **Node** data type is defined as:

```
struct Node {
    int item;
    Node* next;
}
```

and that **first** is a variable of type **Node** that refers to one node in a circular linked list. Let **x** be a variable that refers to a newly created **Node**.



```
Node* x = new Node {4, nullptr};
```

For each of the **independent** code fragments below, fill in the circle that describes its effect. Fill in “neither” if the code invalidates the property that **first** refers to a node in a circular linked list. *Correct answers are worth 1 point; incorrect answers are worth -1 point; blank answers are worth 0 points.*

	inserts x after first	inserts x before first	neither
<code>x->next = first;</code> <code>first = x;</code>	☒	☐	☐
<code>Node *z = first;</code> <code>while(z->next != first)</code> <code>z = z->next;</code> <code>z->next = x;</code> <code>x->next = first;</code>	☐	☒	☐
<code>first->next = x;</code> <code>x->next = first->next;</code>	☐	☐	☒
<code>x = first;</code> <code>x->next = first;</code>	☐	☐	☒
<code>first->next = x;</code>	☐	☐	☒
<code>Node *z = first;</code> <code>x->next = first->next;</code> <code>z->next = x;</code>	☒	☐	☐
<code>x->item = first->item;</code> <code>first->item = x->item;</code> <code>first->next = x;</code>	☐	☐	☒

Question 3: Implementing ADTs12 marks

- (a) [6 marks] The following data type **decimal** represents unsigned integers with configurable accuracy (**precision** is the number of digits). The constructor and **print** function show you, how the numbers are stored internally. It is your task to understand the two functions and implement the **+=** operator below.

```

const int precision = 100;
class decimal {
    std::vector<int> digits;
public:
    // converts an unsigned value to a decimal
    decimal (unsigned value) : digits (precision) {
        for (int i = 0; i != precision; ++i) {
            digits[i] = value % 10;
            value /= 10;
        }
    }
    // prints decimal value
    void print (std::ostream& os) const {
        int i = precision - 1;
        // find first non-zero digit
        while (!digits[i] && i) --i;
        // print in reversed order
        do os << digits[i]; while (i--);
    }
    // adds value of other decimal ignoring overflow
    decimal& operator += (const decimal& other) {

```

Solution:

```

    if (this->digits.size() < other.digits.size())
        this->digits.resize(other.digits.size(), 0);
    int rem = 0, i;
    for (i = 0; i < other.digits.size(); i++) {
        int sum = this->digits[i] + other.digits[i] + rem;
        this->digits[i] = sum % 10;
        rem = sum / 10;
    }
    while (rem > 0) {
        if (i >= this->digits.size()) {
            this->digits.push_back(rem);
            rem = 0;

```

```

    } else {
        int sum = this->digits[i] + rem;
        this->digits[i] = sum % 10;
        rem = sum / 10;
    }
    i++;
}

```

```

        return *this;
    }
}; // end of class decimal

std::ostream& operator << (std::ostream& os, const decimal& value) {
    value.print(os); return os;
}

int main () {
    decimal x = 123;
    x += 1056;
    std::cout << x << '\n';
}

```

- (b) [6 marks] The data type **PQ** in the following program text is supposed to store values in incrementally sorted order. Moreover, values that had been entered multiple times should only be provided once. Implement the member function **PQ::put** accordingly. Example of a program run (0 terminates the input):

```

Input : 1 8 2 7 2 3 1 9 3 3 2 0
Output: 1 2 3 7 8 9

```

```

#include <iostream>

struct Node {
    int value;
    Node* next;
    Node (int value, Node* next): value(value), next(next) {}
};

class PQ {
    Node* first;
public:
    PQ(): first(0) {}
    // retrieve first (and smallest) stored element
    bool get(int& value) {

```

```

        if (first) {
            value = first->value;
            Node* next = first->next;
            delete first;
            first = next;
            return true;
        }
        return false;
    }

    void put(int value); // declaration of put function
}; // end of class PQ

int main() {
    PQ q;
    int value;
    while (std::cin >> value && value > 0)
        q.put(value);
    while (q.get(value))
        std::cout << value << " ";
}

// insert element, value is contained in list once.
// List is sorted in ascending order.
void PQ::put(int value) {

```

Solution:

```

    if (!first) {
        first = new Node{value, nullptr};
    } else if (first->value > value) {
        first = new Node{value, first};
    } else if (first->value != value) {
        Node *cur = first;
        while (cur->next && cur->next->value < value)
            cur = cur->next;
        if (cur->next && cur->next->value == value)
            return; // value already in the list
        cur->next = new Node{value, cur->next};
    }

```

```

}

```

Question 4: Inheritance 20 marks

Publication Management System: You are tasked with developing a simple publication management system for a publishing company. The system will handle *books* and *tapes*, each represented by their respective classes derived from the **Publication** base class.

- **Publication** class: Create a base class Publication with data members title (**string**) and price (**float**). Implement a virtual function **bool isOversize()const** that returns **false** by default.

Solution:

- **Book** class:
 - Derive a class **Book** from **Publication**.
 - Add a data member **pageCount** (**int**) to represent the number of pages in the book.
 - Implement the virtual function **bool isOversize()const** to check if the book is oversize (more than 800 pages).
 - Implement **void getdata()** and **void putdata()** functions to input and display book data.

Solution:

- **Tape** class:
 - Derive a class **Tape** from **Publication**.
 - Add a data member **playingTime** (**float**) to represent the playing time of the audio tape in minutes.
 - Implement the virtual function **bool isOversize()const** to check if the tape is oversize (more than 90 minutes).
 - Implement **void getdata()** and **void putdata()** functions to input and display tape data.

Solution:

Write a `main()` program that creates an array of pointers to **Publication** objects. Ask the user for data about a particular book or tape, and dynamically create an object of the appropriate type. Then add the pointer to the object in the array. After data entry is complete display the resulting data for all the books and tapes entered. Ensure proper memory cleanup by deleting dynamically allocated objects.

Solution: